



# Cambricon-SR: An Accelerator for Neural Scene Representation with Sparse Encoding Table

Tianbo Liu  
USTC  
Hefei, China  
SKLP, ICT, CAS  
Beijing, China  
liutianbo@mail.ustc.edu.cn

Xinkai Song  
SKLP, ICT, CAS  
Beijing, China  
songxinkai@ict.ac.cn

Zhifei Yue  
USTC  
Hefei, China  
SKLP, ICT, CAS  
Beijing, China  
zhifeiyue@mail.ustc.edu.cn

Rui Wen  
SKLP, ICT, CAS  
Beijing, China  
UCAS  
Beijing, China  
wenrui24s@ict.ac.cn

Xing Hu  
SKLP, ICT, CAS  
Beijing, China  
huxing@ict.ac.cn

Zhuoran Song  
SJTU  
Shanghai, China  
songzhuoran@sjtu.edu.cn

Yuanbo Wen  
SKLP, ICT, CAS  
Beijing, China  
wenyuanbo@ict.ac.cn

Yifan Hao  
SKLP, ICT, CAS  
Beijing, China  
haoyifan@ict.ac.cn

Wei Li  
SKLP, ICT, CAS  
Beijing, China  
liwei@ict.ac.cn

Zidong Du  
SKLP, ICT, CAS  
Beijing, China  
duzidong@ict.ac.cn

Rui Zhang  
SKLP, ICT, CAS  
Beijing, China  
zhangrui@ict.ac.cn

Jiaming Guo  
SKLP, ICT, CAS  
Beijing, China  
guojiaming@ict.ac.cn

Di Huang  
SKLP, ICT, CAS  
Beijing, China  
huangdi@ict.ac.cn

Shaohui Peng  
ISCAS  
Beijing, China  
pengshaohui@iscas.ac.cn

Guangzhong Sun  
USTC  
Hefei, China  
gzsun@ustc.edu.cn

Qi Guo\*  
SKLP, ICT, CAS  
Beijing, China  
guoqi@ict.ac.cn

Tianshi Chen  
Cambricon Technologies  
Beijing, China  
tchen@cambricon.com

## Abstract

Neural Scene Representation (NSR) is a promising technique for representing real scenes. By learning from dozens of 2D photos captured from different viewpoints, NSR computes the 3D representation of real scenes. However, the performance of NSR processing running on GPU is insufficient for applications. Cambricon-R achieves high performance of more than 60 scenes per second, but at the cost of modeling quality.

\*Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License.  
ISCA '25, Tokyo, Japan  
© 2025 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-1261-6/25/06  
<https://doi.org/10.1145/3695053.3731018>

In this paper, we propose Cambricon-SR, an algorithm-hardware co-designed accelerator to improve NSR performance and quality. First, we propose a novel NSR algorithm with sparse encoding table, it achieves more than 80% of sparsity for the encoding table with negligible accuracy loss. Second, to efficiently eliminate the invalid memory access requests to the pruned entries, we propose the sparse index unit based on sequential SRAM access to address the challenge of irregular memory access to the sparse structure bitmap. It achieves a throughput increase of 7.54× for the encoding stage with an additional area overhead of 8.59%. Third, we propose the dynamic shared buffer for the MLP units to reduce the buffer usage by 85.3% so that we can increase the number of the MLP units.

We conduct experimental evaluations on 8 typical scenes. The results demonstrate that, compared with A100 GPU and Cambricon-R, Cambricon-SR achieves 1259× and 4.12× speedup, and reduces

the energy consumption by 1139× and 2.98× for each training iteration of NSR, respectively.

## CCS Concepts

• **Computer systems organization** → **Architectures**.

## Keywords

neural scene representation, sparse encoding table, accelerator

### ACM Reference Format:

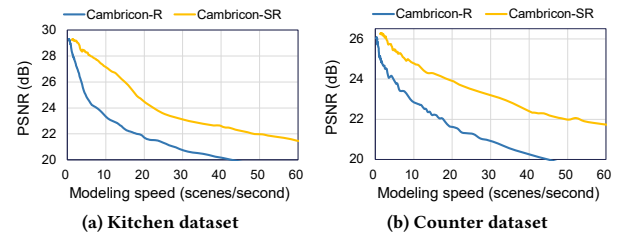
Tianbo Liu, Xinkai Song, Zhifei Yue, Rui Wen, Xing Hu, Zhuoran Song, Yuanbo Wen, Yifan Hao, Wei Li, Zidong Du, Rui Zhang, Jiaming Guo, Di Huang, Shaohui Peng, Guangzhong Sun, Qi Guo, and Tianshi Chen. 2025. Cambricon-SR: An Accelerator for Neural Scene Representation with Sparse Encoding Table. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3695053.3731018>

## 1 Introduction

Neural scene representation (NSR) is a novel algorithm to reconstruct 3D representation of real scenes. It learns the encoded 3D representation from dozens of 2D photos taken from RGB cameras. The 3D information can then be used for novel view rendering [25, 27, 29], scene editing [2, 17, 53], relighting [43, 47, 54], and so on. Compared to traditional 3D reconstruct technique, it avoids the substantial human efforts and expensive laser cameras required for 3D representation, significantly reducing the cost and time for realistic 3D representation.

NSR can be applied to various fields such as autonomous driving [6, 15, 51, 52], embodied intelligence [1, 5, 7, 18, 30, 56], and AR/VR [10, 23]. However, the insufficient modeling performance of NSR limits its application. In many real-world applications, the surrounding environment is changing, requiring continuous NSR modeling of the environment. For instance, in autonomous driving, modeling the environment requires low latency. Due to the insufficient modeling performance of NSR, current NSR can only model environmental information offline for data generation and driving simulation, and cannot be used for modeling the environment online [6, 15, 48, 51]. Similarly, in embodied intelligence, using NSR for scene reconstruction can help robots understand and interact with the scene [5, 7, 30]. However, the NSR modeling process in these applications is often completed offline in advance [1, 18, 56], thus only supporting pre-determined static scenes. These applications set high demands on both the performance and quality of NSR.

However, the performance of NSR running on GPUs is insufficient. For example, it takes at least 5 seconds to compute the representation of a scene on A100 [36], which cannot meet the requirement of the aforementioned applications. Several accelerator architectures have been proposed to speed up NSR processing [24, 42, 55]. Accelerators like Instant-3D [24] and Instant-NeRF [55] are designed for edge devices. They improve the NSR performance by around 200 times compared with edge-end GPUs and achieve NSR performance of around 1 scene per second. Other accelerators like Cambricon-R [42] are implemented on a large chip. By keeping the encoding table on chip, almost all off-chip memory accesses are eliminated. They achieve NSR performance of more than 60 scenes per second. However, the high performance of Cambricon-R is



**Figure 1: Performance-quality curves of Cambricon-R [42] and Cambricon-SR on Kitchen and Counter datasets. The performance-quality curves demonstrate the trade-off between the modeling speed and quality. By reducing the number of training iterations, the modeling speed can be improved with the damage of modeling quality. The x-axis represents the modeling speed through the number of scenes per second. The y-axis represents the modeling quality through PSNR (Peak Signal-to-Noise Ratio). The curve closer to the top right corner indicates better performance and quality.**

achieved at the cost of modeling quality. The NSR process involves performing a training task for each frame of the scene. Reducing the number of training iterations per frame improves performance but degrades the quality of the trained model. Cambricon-R only completes 250 iterations in training of each scene, which is insufficient for representations of large-scale real-world scenes. For instance, in the large-scale dataset Kitchen, training for 250 iterations only achieves a PSNR of 18.16. At this level of quality, the results are severely affected by noise, making them impractical for applications. Figure 1 shows the trade-offs between performance and quality in two typical large-scale real-world scenes.

In this paper, we aim to improve both the performance and quality of NSR accelerator with a key technology of sparse encoding table. We observe that a large number of the entries in the encoding table can be pruned with negligible impact to the representation quality of NSR, as shown in Figure 4. When using the sparse encoding table, memory accesses to the pruned entries can be avoided to speed up the encoding stage, which is bottleneck stage of NSR due to the heavy fine-grained irregular memory accesses even for on-chip memory [42].

Based on this observation, we first propose a training algorithm called sparse table NSR (ST-NSR), which achieves more than 80% sparsity of the table during the forward stage and more than 90% sparsity of the table gradient during the backward stage at the cost of negligible accuracy loss. To the best of our knowledge, this is the first NSR training algorithm with sparse encoding tables. We observe that as the size of the encoding table increases, the sparsity rate grows, resulting in a reduction of required memory accesses, as shown in Figure 3. However, a larger encoding table comes at the cost of an increased memory footprint, making it impossible to store the entire table on-chip. Based on the design space exploration of different table sizes, we determine that a table size of 128 MB maximizes the number of invalid memory access requests while ensuring that the sparse table can still be stored on-chip.

In the update stage of ST-NSR, a dense table must be maintained to accumulate gradients. Due to the large size of the dense table, it must be stored off-chip. As a result, the large amount of off-chip memory access to this table in the update stage becomes a performance bottleneck. To address this challenge, we propose sparse update unit that efficiently updates the sparse table on-chip with sparse storage format, thus avoiding off-chip memory access to the dense table and reducing the memory access volume by 83.5% compared to the straightforward approach.

Efficient memory access to the on-chip sparse table is challenging. During the forward stage, many invalid memory requests occur for pruned entries. Efficiently eliminating these requests before accessing the sparse table is difficult due to the irregular access patterns of the sparse structure bitmap. We propose a Sparse Index Unit (SIU) based on sequential SRAM accessing and parallel address matching to address this challenge. With an additional area overhead of 8.59%, SIU achieves high throughput filtering of memory access requests, thereby increasing the throughput of encoding stage by 7.54×.

After improving the throughput of the encoding stage, the MLP stage becomes a new bottleneck of ST-NSR algorithm. However, due to the large local buffer used by each MLP unit, increasing the MLP units by 4× would require an additional total buffer capacity of 144 MB, significantly raising the cost of the chip. We propose a design with dynamic shared buffers that reduces the average buffer capacity required per MLP unit by 85.3%. As a result, we can increase the number of MLP units by 4× to improve performance with acceptable size of buffer requirement.

We conduct experimental evaluations on 8 typical scenes. The results show that, compared with A100 GPU and the state-of-the-art NSR training accelerator Cambricon-R, Cambricon-SR achieves 1259× and 4.12× speedup, and reduces the energy consumption by 1139× and 2.98× for each training iteration of NSR, respectively. Thus, we improve both the performance and quality of NSR. As shown in Figure 1, under the same modeling speed, the quality improvement comes from more iterations of training.

The contributions of this paper are summarized below:

- We propose the first NSR algorithm with sparse encoding table (ST-NSR). It achieves more than 80% sparsity of the encoding table and its gradients. We implement ST-NSR on GPU and it achieves a 1.55× speedup in the encoding stage and a 1.19× overall speedup.
- We propose the Sparse Update Unit to reduce 83.5% of the off-chip memory accesses in the update stage. It decreases the execution time for the update stage by 5.28×.
- We propose the Sparse Index Unit (SIU) based on sequential SRAM access to address the challenge of irregular memory access to the bitmap. It achieves a throughput increase of 7.54× during the encoding stage with an additional area overhead of 8.59%.
- We propose the dynamic shared buffer for the MLP units to reduce the buffer usage by 85.3% so that we can increase the number of the MLP units by 4× with acceptable capacity of on-chip buffers.
- We conduct thorough experimental evaluation of Cambricon-SR. The results show that, compared with A100 GPU and the state-of-the-art NSR training accelerator Cambricon-R,

Cambricon-SR achieves 1259× and 4.12× speedup, and reduces the energy consumption by 1139× and 2.98× for each training iteration of NSR, respectively.

## 2 Background and motivation

### 2.1 NSR Algorithms

NSR algorithms are primarily divided into two main categories: NeRF-based [33] and 3DGS-based [19]. In this paper, we focus on NeRF-based NSR algorithms. 3DGS-based algorithms explicitly represent the scene using millions of discrete Gaussians in 3D space. In contrast, NeRF-based algorithms learn implicit representation of a scene by sampling points in 3D space and using MLPs to learn the mapping between spatial position of points and their color and volume density. However, due to the large size of the MLP, the modeling speed is exceedingly slow, making it difficult to meet the demands of applications.

To improve modeling efficiency, optimized NSR algorithms combine encoding table with small MLPs. Each point is obtained through a lookup table to get encoded features, which are then processed by small MLPs. This method significantly reduces the computational workload for each point. Currently, many mainstream NSR algorithms [4, 9, 12, 36, 46, 49] adopt this technique of combining encoding table with small MLPs. A training iteration of the algorithms consists of following stages, as illustrated in Figure 2:

**(1) Sampling.** In the sampling stage, pixels are randomly sampled from the training images. The coordinates and the RGB color of sampled pixels are the input and the ground truth of the NSR process, respectively. To compute the color of pixels, each pixel is mapped to a ray [33, 36] or a cone [3, 4], which starts from the camera viewpoint through the corresponding pixel along the viewing direction. Then, points are randomly sampled along each ray or cone. The coordinates  $(x, y, z)$  and view direction  $(\theta, \phi)$  of these sampled points are computed and transmitted to the next stage.

**(2) Encoding.** Hash grid [36] and Multi-planes [12] are two mainstream encoding methods of NSR. The process of hash grid is shown in the middle part of Figure 2. **Step ① hash encoding of vertices:** Specifically, For a given sampled point  $s$ , the voxel that encloses the point is determined, and its 8 corner vertices are identified. Through the hash function given in Equation 1, the vertices' coordinate  $(x_i, y_i, z_i)$  is mapped to the hash table index  $h_i$ . **Step ② encoding table lookup:** Using the index  $h_i$ , an  $F$ -dimensional feature vector is retrieved for each vertex of the voxel from a compact encoding table containing  $T$  entries. **Step ③ interpolation & concatenation:** Linear interpolation of the 8 vertices'  $F$ -dimensional feature vectors yields an  $F$ -dimensional feature vector. This process is repeated across  $L$  different grid resolutions using the corresponding hash tables. The feature vectors from all levels are then concatenated to construct an input vector of size  $F \times L$  for the MLP.

$$h_i = H(x_i, y_i, z_i) = (\pi_1 x_i \oplus \pi_2 y_i \oplus \pi_3 z_i) \mod T^1 \quad (1)$$

Note that each resolution level has an independent hash table, where each hash table consists of  $T$  entries, and each entry stores

<sup>1</sup>  $\pi_1 = 1$ ;  $\pi_2 = 2\,654\,435\,761$ ;  $\pi_3 = 805\,459\,861$

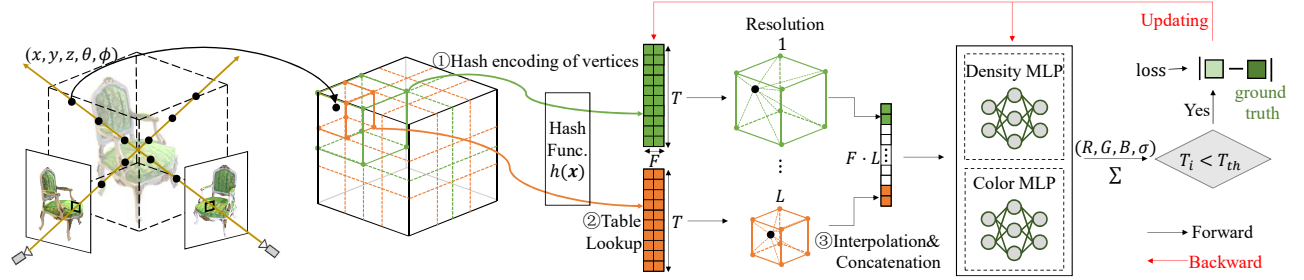


Figure 2: The training process of a typical NSR algorithm. Hash grid encoding is used in this example.

an  $F$ -dimensional feature vector. Therefore, the total number of trainable parameters in the hash encoding tables is  $L \times T \times F$ .

In multi-planes encoding [12], sampled points are first projected to multiple planes (x-y plane, x-z plane, and y-z plane in case of 3D space). Each plane corresponds to a series of encoding tables at different levels of resolution. Similar with hash grid, the feature vector of a point is computed through bilinear interpolation of 4 vertices of the grid that surrounds the point at each level. Then, features of different planes from the same level are reduced using operations such as element-wise multiplication.

(3) **MLP.** In the MLP stage, the encoded feature vector of each sampled point is fed into MLPs to predict the RGB color and volume density of the point.

(4) **Volume Rendering & Loss Computing.** Volume rendering is performed to accumulate the colors and densities of sampled points along each ray, resulting in the color of the corresponding pixel. Then, the computed pixel color is compared with the ground truth color to calculate the loss.

(5) **Parameters Updating.** After computing the loss, gradients are back-propagated through the above pipeline. The learnable parameters in NSR, primarily including entries in encoding tables and weights of MLPs, are optimized using optimization algorithm such as gradient descent.

## 2.2 NSR Accelerators

Some applications of NSR algorithm, such as autonomous driving and embodied intelligence, require a high 3D representation performance. However, executing NSR processing for a typical scene takes at least 5 seconds on A100 to achieve an acceptable representation quality, which cannot meet the requirement of these applications. Some researchers have proposed designs for NSR accelerators to address the performance gap.

**Instant-NeRF.** Instant-NeRF [55] accelerates NSR training using a Near Memory Processing (NMP) architecture. To address the memory access bottleneck in the NSR processing, Instant-NeRF integrates computing units directly on the memory, achieving greater bandwidth.

**Instant-3D.** Instant-3D [24] splits the hash grid into two branches and reduces the grid size and update frequency for the less sensitive branch. It also minimizes memory read/write operations during hash grid interpolation by employing memory access merging.

**NeuGPU.** NeuGPU [40] is an NSR training accelerator chip that has taped out. It partitions the encoding table for better spatial

locality, and skips the access of encoding table when the sampled point is far from the grid vertices.

These three accelerators are designed for edge devices and achieve huge improvement against edge-end GPUs. However, due to the limited hardware resources of edge-end chips, high performance NSR processing cannot be achieved by these accelerators.

**Cambricon-R.** Cambricon-R [42] introduces a fully fused architecture to address the memory access challenges in NSR. It proposes a ray-based execution flow which keeps almost all data on-chip, thereby eliminating the majority of off-chip memory accesses. Additionally, it introduces a network-on-chip (NoC) to alleviate the fine-grained irregular memory access conflicts during encoding table access, thus resolving the memory access challenges during the encoding stage. As a result, Cambricon-R achieves high performance NSR processing for the first time with on-chip storage resources comparable to those of an A100.

However, Cambricon-R has insufficient training iterations under high performance requirements. The NoC for accessing encoding table has limited throughput. When improving the throughput of the NoC, the additional area cost of wires and buffers will increase rapidly as the number of banks increases. Due to insufficient throughput, Cambricon-R can only complete 250 training iterations for each scene, which seriously damages the modeling quality, especially for large-scale scenes.

## 3 ST-NSR Algorithm

Multiple works [21, 24, 42, 55] have demonstrate that a large number of irregular table accesses during the encoding stage are a performance bottleneck of NSR process. In order to reduce the number of encoding table access, we adopt a sparse algorithm ST-NSR to prune the encoding table and its gradients during training.

### 3.1 Process of ST-NSR

We propose ST-NSR, the first NSR training algorithm with sparse encoding tables to achieve high sparsity with negligible accuracy loss. Algorithm 1 illustrates the ST-NSR process, where sparsification-related operations are marked in red. Note that the original dense encoding table (DT) is defined as  $T_{\text{dense}}$ , while the pruned sparse encoding table (ST) is defined as  $T_{\text{sparse}}$ , which is stored using a sparse storage format. ST-NSR consists of the following three stages:

(1) **Forward stage, Line 3 – 6.** In the forward stage, the process of the ST-NSR algorithm is the same as that of the NSR algorithm, involving sampling, encoding, MLP, and loss computation. The

**Algorithm 1** ST-NSR Training Algorithm

---

**Data:**  $T_{\text{dense}}$ : dense encoding tables.  $T_{\text{sparse}}$ : sparse encoding tables.  
 $W_{\text{MLP}}$ : weights of MLPs.  $N_{\text{iter}}$ : the number of training iterations.  
 $\theta$ : the sparsity threshold.

```

1: for  $i = 1, 2, \dots, N_{\text{iter}}$  do
2:    $dT_{\text{accum}}, dW_{\text{accum}} \leftarrow 0$ 
3:   for each  $\text{ray}$  in Sampling(training-data) do
4:      $\text{features} \leftarrow \text{Encoding}(\text{ray}, T_{\text{sparse}})$ 
5:      $\text{color} \leftarrow \text{MLP}(\text{features}, W_{\text{MLP}})$ 
6:      $dc \leftarrow \text{Loss}(\text{color}, \text{color}_{\text{ground\_truth}})$ 
7:      $df, dW \leftarrow \text{Backward-MLP}(dc, W_{\text{MLP}})$ 
8:      $df_s \leftarrow \text{Backward-Sparsification}(df)$ 
9:      $dT \leftarrow \text{Backward-Encoding}(df_s)$ 
10:     $dT_{\text{accum}} \leftarrow dT_{\text{accum}} + dT$ 
11:     $dW_{\text{accum}} \leftarrow dW_{\text{accum}} + dW$ 
12:  end for
13:   $T_{\text{dense}} \leftarrow \text{Parameters-Updating}(T_{\text{dense}}, dT_{\text{accum}})$ 
14:   $W_{\text{MLP}} \leftarrow \text{Parameters-Updating}(W_{\text{MLP}}, dW_{\text{accum}})$ 
15:   $\theta \leftarrow \text{Threshold-Updating}(T_{\text{dense}})$ 
16:   $T_{\text{sparse}} \leftarrow \text{Forward-Sparsification}(T_{\text{dense}}, \theta)$ 
17: end for

```

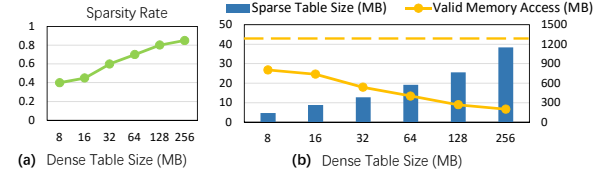
---

difference lies in the encoding stage, where ST-NSR algorithm accesses the pruned ST instead of the original DT. Since most of entries are pruned, a large number of invalid memory accesses can be eliminated. The generation of ST will be introduced in (3) **Update stage**.

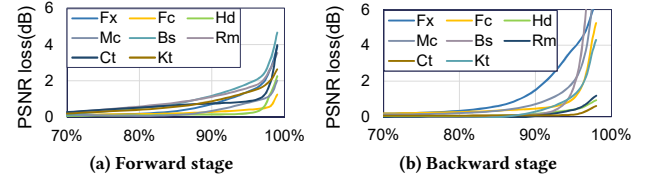
(2) **Backward stage**, Line 7 – 11. First, the loss gradient is back-propagated through the MLP, obtaining gradients for both the MLP weights and the encoded features. A dedicated backward sparsification step (Line 8) is designed to sparsify the gradients of the encoded features ( $df$ ). Specifically, for each gradient vector, if the absolute values of all its dimensions are below the sparsity threshold, the vector is pruned. Subsequently, the remaining gradients ( $df_s$ ) are further propagated to the encoding stage, and the accumulated gradients for both the MLP and encoding are updated.

(3) **Update stage**, Line 13 – 16. During the update stage, using the accumulated gradients computed in the backward stage, entries in the DT and weights in the MLPs are updated through optimization algorithms. Subsequently, given a pre-determined sparsity rate, the sparsity threshold  $\theta$  is dynamically determined using the top-k algorithm based on the updated DT. In the DT, entries with all dimensions below  $\theta$  are pruned, while others are retained, generating the ST for the next training iteration.

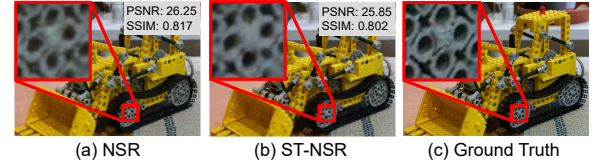
In each iteration of the ST-NSR, which entries in the encoding table are pruned or retained changes dynamically. To achieve this, gradients must be continuously accumulated for all entries until their accumulated values exceed or fall below a threshold, triggering changes in sparsity. However, since ST only contains the active entries that have not been pruned in the current iteration, it cannot fulfill this requirement. Therefore, it is necessary to maintain a DT that includes all entries to accumulate gradients. This ensures that pruned parameters still have a chance to be restored rather than being permanently discarded due to temporary inactivity. For example, in Figure 9, the first entry in the DT gradually accumulates from 0.1 to 6, eventually becoming a valid parameter.



**Figure 3: (a) The impact of dense table size on sparsity rate. (b) The impact of dense table size on sparse table storage and volume of valid memory access. The yellow dashed line represents the total volume of memory access.**



**Figure 4: The impact of sparsity on PSNR loss.**



**Figure 5: Modeling quality comparison of NSR and ST-NSR**

### 3.2 Sparsity-Size Trade-off

We observe that the size of the dense table (DT) has a significant impact on sparsity rates. We evaluate the sparsity rates under different table sizes, ensuring that the PSNR degradation does not exceed 0.5. The experimental results are shown in Figure 3. As the size of DT increases, higher sparsity rates are achieved. Since the total volume of accesses to the tables is same across different DT sizes, the volume of valid accesses is reduced, thereby creating potentials for performance improvement. However, the increase in the size of DT also results in a larger ST. To reduce the volume of valid accesses by increasing the DT size while ensuring that the ST can be stored on-chip, we select the DT size to 128 MB. Under this configuration, the size of the sparse table is no more than 26 MB. And according to our evaluations, the size of table gradients is no more than 14 MB in this case, meaning that sparse table and table gradients can be stored in a 40 MB on-chip buffer.

**Sparsity Evaluation.** In ST-NSR, different sparsity rates are set for the encoding tables in the forward stage and for the gradients of encoded features in the backward stage. Under the aforementioned configuration of table size, we conduct experimental evaluations on two types of sparsity in various datasets. Figure 4a shows the impact of sparsity in the forward stage on the modeling quality (measured in PSNR), while the sparsity in the backward stage is set to 0. The results indicate that when the sparsity rates for 8 typical scenes are set to 86%, 88%, 96%, 94%, 82%, 80%, 84%, and 82%, respectively, the



impact on PSNR is less than 0.5. Figure 4b illustrates the impact of sparsity in the backward stage on the modeling quality, with the forward stage sparsity set according to the results mentioned above. The results demonstrate that when the sparsity rates for 8 typical scenes are set to 80%, 85%, 95%, 85%, 85%, 90%, 95%, and 85%, respectively, the impact on PSNR is less than 0.5. As shown in Figure 5, a PSNR reduction of around 0.5 has little impact on the visual quality. The remainder of this paper will be based on these sparsity results.

### 3.3 GPU Optimization

According to our evaluation of Instant-NGP [37], a representative NSR algorithm, the encoding stage is most time-consuming, accounting for 46% of the total runtime. This is due to massive fine-grained irregular memory accesses to encoding tables during the encoding stage. In this section, we implement ST-NSR algorithm on NVIDIA A100 GPU using CUDA to reduce the volume of memory accesses and improve the performance of the encoding stage.

**We first attempt to leverage the structured sparsity hardware of the A100 to support ST-NSR, but find it to be completely ineffective.** The structured sparsity in the A100's Tensor Cores, such as 2:4 sparsity, is designed to accelerate matrix multiplications in NNs by reducing MAC operation overhead [34]. In contrast, ST-NSR leverages sparsity to reduce memory access in the encoding stage, which does not involve any matrix multiplication operations, making structured sparsity hardware ineffective for ST-NSR. **Moreover, applying the structured sparsity algorithm to sparse encoding table is also ineffective.** We apply structured sparsity to the encoding table with the same sparsity rate as ST-NSR. Experiments show that it not only fails to improve performance but also degrades quality, reducing PSNR by 1.5 dB. This is because structured sparsity (e.g., 2:4) requires accessing four consecutive entries as a group to gain performance benefits. However, accesses to the encoding table are irregular single-entry lookups, resulting in no performance gain. Moreover, the fixed sparsity pattern further degrades modeling quality.

In our GPU implementation of ST-NSR, sparse table (ST) is kept resident in the L2 cache to reduce off-chip memory accesses, as its size is less than 40 MB. To further filter out invalid ST accesses, a bitmap stored in shared memory is utilized to track the sparsity state of each table entry. Since each block in our CUDA kernel only processes one level of encoding table, we only load the bitmap corresponding to that level into shared memory. However, shared memory on A100 (164 KB per block) cannot accommodate the complete bitmap of one level (256 KB). To address this issue, we employ a hash table, which saves storage to 128 KB through hash mapping at the cost of hash collisions that may cause a small number of pruned entries to be misidentified as valid.

We evaluate the performance of NSR and ST-NSR on GPU across 8 datasets. Figure 6 shows the runtime of one training iteration of NSR and ST-NSR on GPU. ST-NSR achieves a 1.55× speedup in the encoding stage and a 1.19× overall speedup compared to the baseline GPU implementation. However, there remains a significant performance gap between the achieved speedup and the theoretical reduction in memory accesses (at least 5×). This is primarily attributed to bank conflicts issues when threads within a block

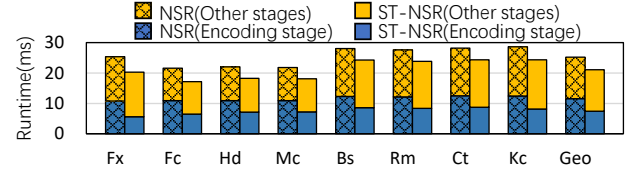


Figure 6: Comparison of NSR and ST-NSR runtime on GPU.

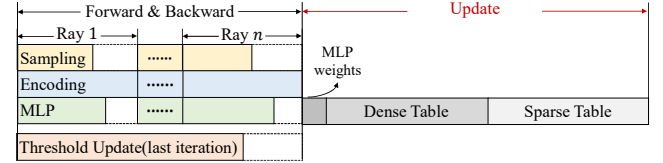


Figure 7: Dataflow of Cambricon-SR for one iteration of NSR training.

concurrently access the bitmap in shared memory, as the access addresses are irregular indexes computed by hash functions. The inefficiency motivates our design of a dedicated hardware accelerator (Section 4) to address these limitations.

## 4 Cambricon-SR

In this section, we present Cambricon-SR, a hardware architecture developed to efficiently execute the ST-NSR algorithm proposed in Section 3. We first introduce the dataflow and overall architecture of Cambricon-SR in Section 4.1 and Section 4.2. Subsequently, we detail the optimizations applied to the update stage, the encoding stage, and the MLP stage, which include Sparse Update Unit in Section 4.3, Sparse Index Unit in Section 4.4, and Dynamic Shared Buffer in Section 4.5, respectively.

### 4.1 Overall Dataflow of Cambricon-SR

Figure 7 shows the high-level execution flow of Cambricon-SR for one iteration of NSR training. We divide the entire execution process into two stages: the forward & backward stage, and the update stage.

**Forward & Backward Stage.** In this stage, by storing all involved parameters and intermediate data on-chip, off-chip memory accesses are nearly eliminated. Weights and gradients of MLP are small and stored on-chip. For parameters of encoding tables, due to ST-NSR algorithm, the total size of the sparse table and table gradients does not exceed 40 MB. Therefore, it is feasible to store them on-chip. The size of the intermediate data depends on the number of rays or sampled points processed in parallel. To reduce the size of intermediate data and store those data on-chip, sampling, encoding, and MLP processes of one ray are executed in a fine-grained pipelined manner, with a granularity of 32 sampled points.

**Update Stage.** During the update stage, three types of model parameters need to be updated: MLP weights, sparse table (ST), and dense table (DT). The MLP weights and gradients are stored on-chip and are updated directly on-chip. Simultaneously, the off-chip DT is updated by using the on-chip table gradients generated at the

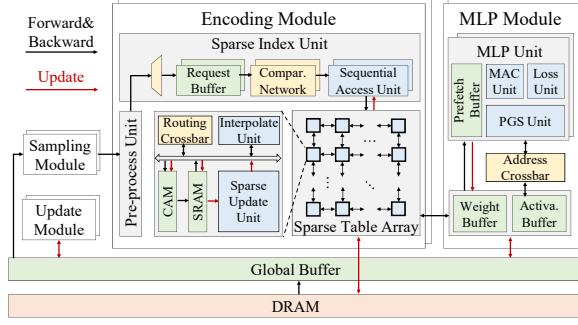


Figure 8: Overall Architecture of Cambricon-SR.

forward & backward stage. Subsequently, the DT is read to compute the sparsity threshold, and then each DT entry is compared with this threshold to update the on-chip ST. The optimization of this process is introduced in Section 4.3.

We observe minor changes in the sparsity threshold between iterations. To avoid waiting for the computing of the sparsity threshold, Cambricon-SR utilizes the sparsity threshold from the last iteration during the update process of the sparse table. Previously, the threshold update process was performed between the DT and ST updates, but our dataflow shifts this operation to run in parallel with the forward and backward stage. However, the computes of threshold takes longer time than the forward and backward process because it requires read the entire DT. We propose imprecise computation of the threshold by using only half of the DT. As a result, the time taken for sparsity threshold computation is shorter than the time for the forward and backward stage. Due to the randomization of DT access addresses through hash functions, we consider the data distribution in DT to be approximately uniform. Therefore, we simply use the first half of the DT to calculate the threshold. Experimental evaluations show that this mechanism of updating threshold has negligible impact on the representation accuracy of ST-NSR (less than 0.1).

## 4.2 Overall Architecture of Cambricon-SR

The overall architecture of Cambricon-SR is optimized for the ST-NSR algorithm based on a fully fused on-chip processing architecture, as illustrated in Figure 8.

In Cambricon-SR, the **Global Buffer** primarily stores the training data for each training iteration, the weights of the MLP, and the accumulated gradients of the MLP. The **Sampling Module** is responsible for performing the conversion from pixels to rays, as well as sampling points along the rays. The **MLP Module** handles the training of the MLP. The MLP module comprises 512 MLP units, each consisting of two  $32 \times 32$  MAC (Multiply Accumulate) arrays. The optimized design of the MLP Module will be discussed in Section 4.5. To enable sparsity in the backward stage of the ST-NSR algorithm, the MLP unit includes a Point Gradients Sparse Unit (PGS Unit), which is designed to prune the gradients of the encoded features obtained during the backpropagation of MLP. The **Update Module** gathers and accumulates the weight gradients from the MLP module, then updates the weights in the global buffer. Based on the architecture of [8], which is constructed with

a bitonic sorter and cascaded bidirectional insertion sorting units, the **Update Module** performs top-k sorting on DT and updates the sparsity threshold.

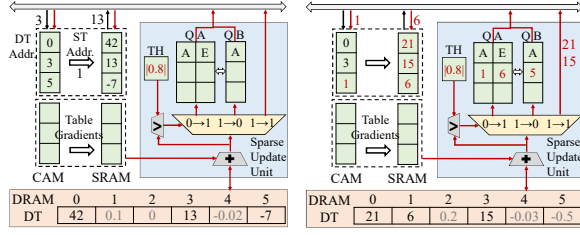
**Encoding Module.** The Encoding Module is responsible for performing memory access to the sparse tables and executing the interpolation for each sampled point. Inside the encoding module, the **Pre-process Unit** is responsible for calculating the addresses of the vertices surrounding the sampled points in the table at each level. The **Sparse Index Unit (SIU)** is responsible for eliminating invalid memory access requests for entries that have been sparsified. Details of SIU will be discussed in Section 4.4.

**Sparse Table Array (STA).** The encoding module is composed of 16 STAs, corresponding to 16 levels of encoding tables, with a total of 40 MB SRAM. The STA performs high-throughput memory access to the sparse table and its gradients. STA is implemented based on the Auto-Interpolation Bank Array of Cambricon-R [42] since it achieves SOTA throughput of encoding in NSR. Specifically, nodes in the array are interconnected in a mesh NoC. Each node contains an SRAM bank that stores a part of sparse tables and gradients. During the encoding process, memory access requests are transmitted between nodes via the NoC. And data access and interpolation computations are performed when the request arrives at the corresponding node. This approach resolves bank conflict issues through asynchronous control of different nodes. Compared to Cambricon-R, we modify the structure of node to efficiently support ST-NSR algorithm. As shown in Figure 8, **CAM** and **Sparse Update Unit** (detailed in Section 4.3) are integrated into the STA.

**Content Addressable Memory (CAM).** To improve space utilization of the STA, we compactly store the valid entries of the sparse table in SRAM. However, each access to an entry in the sparse table (ST) requires the translation of the original address in the dense table (DT) to the address in the sparse table, to correctly access the entry from the on-chip SRAM. We propose the use of CAM to implement this address translation. CAM is a type of memory device that, upon inputting its stored content, returns the corresponding address of the content. Each node in STA has a corresponding CAM to perform address translation. As shown in Figure 9 (left), the entry at DT address 3 is accessed, translated to the corresponding ST address 1 via the CAM, and then the entry 13 at SRAM address 1 is retrieved.

In our hardware configuration, each node in the STA can store up to approximately 2,500 table entries and table gradients in total. To reduce the significant area and energy overhead associated with the parallel match of such a large number of entries in the CAM, we adopt a blocking strategy, a technique commonly used in the design of large CAM [41]. Specifically, each node includes 16 blocks of CAM, each with 160 entries. Each block operates independently, handling search, read, and write operations. When a request with an address from the dense tables arrives at the CAMs. The corresponding block of CAMs is determined by the upper 4 bits of the address, and only the selected block is activated. We evaluate the CAM design using Synopsys Design Compiler. According to our experimental results, the total area of the 15 MB CAM in Cambricon-SR is  $33.56mm^2$ , accounting for 14.29% of the total chip area.

**Off-chip Memory.** We observe that the dense table used in the update stage is not only the largest in size but also accessed



**Figure 9: An example of sparse table (ST) access (left) and update (right) in a node of the Sparse Table Array (STA). For simplicity, the dimension of table entries is 1 in this figure.**

with regular and sequential addresses. Therefore, unlike the sparse tables and its gradients, which are stored in STA, the dense table is stored in off-chip memory.

### 4.3 Sparse Update Unit

According to our evaluation, off-chip memory access to the DT accounts for 72.1% of the total ST-NSR processing time, making it a significant performance bottleneck. In the ST-NSR process, there are two stages that require accessing the off-chip DT. Firstly, during the update of the DT, we observe that the table gradients exhibit a sparsity of at least 89%, meaning that no more than 11% of the DT data needs to be loaded and stored even once. This stage contributes to 13.0% of the total ST-NSR processing time. Secondly, during the update of ST, all entries in the DT are read to update the on-chip ST. Even though DT access is regular and continuous, the large volume of data results in this stage accounting for 59.1% of the processing time.

To alleviate the performance degradation caused by the large volume of off-chip memory accesses, we propose a sparse update strategy to optimize the update process for ST. By comparing the encoding tables of adjacent iterations, we observe that the sparsity of entries tends to stabilize as the training progresses. After each iteration, sparsity of only no more than 0.8% of entries changes, except for the first 20 iterations. This insight inspires us to update the ST incrementally.

However, when updating the ST, some new entries, whose absolute values after updating exceed the sparsity threshold, need to be inserted into the ST. Similarly, some entries whose values fall below the sparsity threshold need to be deleted from the ST. This process is challenging because the on-chip ST is stored compactly. The straightforward implementation involves first identifying the entries to be deleted, shifting all subsequent entries in the compact storage to fill the gap created by the deletion, and then finding the appropriate position for the new entries to be inserted. This requires a significant amount of data movement in the storage of the ST, leading to severe performance degradation.

To address this issue, we design a sparse update unit based on out-of-order storage and in-place data replacement, as shown in Figure 9. During the update of DT, Cambricon-SR computes the new value of changed entries. Then, it compares the values before and after the update with the sparsity threshold to classify changed entries into 4 categories according to the sparsity state of each changed entry. For entries changing from sparse to sparse, Cambricon-SR skips

them as no on-chip updating of ST is required. For entries changing from dense to dense, as these entries are already stored on-chip, Cambricon-SR directly write the updated value into ST ("21" and "15" in Figure 9). This process is performed simultaneously with the update of the DT. For entries changing from sparse to dense, the address in DT and the updated value are stored into Queue A in the Sparse Update Unit. For entries changing from dense to sparse, the address in DT is stored into Queue B in the entry update.

In each STA node, one data is read from each of the two queues simultaneously. From Queue A, an address (Address A, "1" in Figure 9) and an entry (Entry A, "6" in Figure 9) are read. From Queue B, an address (Address B, "5" in Figure 9) is read. First, Address B is translated into the corresponding address in ST (Address C, "2" in Figure 9) through CAM. Then, the Address A is write into the Address C of the CAM and the Entry A is write into the Address C of the SRAM bank for sparse entries. This mechanism ensures that the space released by an old entry (stored at address B in the node) is directly utilized to store the new entry, and the address data in CAM is updated accordingly.

Since the sparsity threshold is updated based on the predetermined sparsity rate, the number of entries in the two queues is roughly equal. For minimal extra entries (around 0.03%), if the entry is in Queue B, it is removed from the CAM and SRAM, and the last entry is moved to its position. If the entry is in Queue A, it is added to the end of the CAM and SRAM. Note that the capacity of the CAM and SRAM in Cambricon-SR is redundant to accommodate such minimal extra entries. As a result, Cambricon-SR avoids the overhead of extensive data movement, achieving efficient update of ST.

Through the aforementioned optimizations of sparse table update, Cambricon-SR reduces the total memory accesses during the table update process by 6.06×. The overall execution time for the update stage has decreased by 5.28×, and the proportion of time spent on the update stage to the total time has decreased to 32.85%.

### 4.4 Sparse Index Unit

The ST-NSR algorithm significantly reduces the number of valid table entries that need to be stored. However, due to the hash function in the encoding stage, the mapping between grid vertices and table entry addresses is irregular, making it difficult to determine which entries are pruned during memory access request generation. Due to the high sparsity of encoding tables, a large portion of memory access requests target invalid addresses, leading to wasted bandwidth in the sparse table array and severe congestion in the NoC. To improve memory access throughput in ST-NSR, invalid memory access requests must be identified and eliminated before reaching the sparse table array. We introduce the sparse index unit (SIU) to efficiently handle this problem.

**4.4.1 Challenges.** The throughput requirement of SIU is estimated according to the typical sparsity of around 86% and the throughput of the sparse table array of 256 requests per cycle. Thus, the SIU need to process around 1800 memory requests per cycle to fully utilize the throughput of the sparse table array. And we choose 2048 as the target of SIU throughput.

A straightforward implementation of SIU is to parallelly accessing the bitmap that stores the sparsity of each entry in a bit.



However, it requires to execute 2048 irregularly distributed single-bit memory access per cycle. A naive implementation is to first split the bitmap into 2048 banks and connects all input addresses and all banks through a crossbar. However, the  $2048 \times 2048$  crossbar will cause substantial wiring congestion and area overhead. In our experiments, EDA tools are unable to complete the layout and routing for it. Even if the crossbar is implemented, it still faces severe bank conflict problem. Since the addresses for the bitmap are generated by the hash function, the addresses are randomly distributed across 2048 banks. The number of requests that each bank needs to handle per cycle is unbalanced, requiring waiting for the busiest bank to complete all of its accesses. This results in severe throughput degradation of 83%.

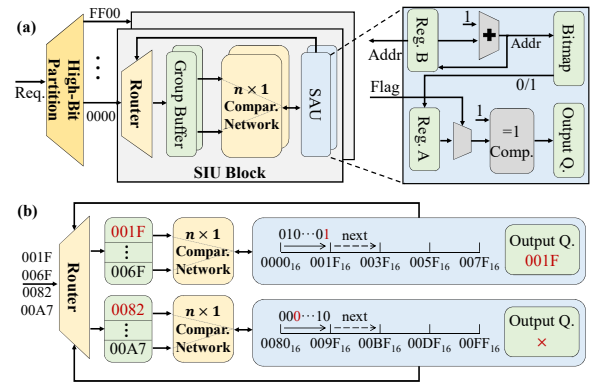
**4.4.2 Our Design.** To address the issues of complex routing and bank conflicts caused by high-throughput random access to SRAM, we propose a method based on sequential access to SRAM. We notice that these issues come from the irregular memory access to the SRAM. So we propose an idea to first sequentially and iteratively read words from the SRAM, and then match the word with the input addresses.

**Sequential Access.** As illustrated in Figure 10(a), Sequential Access Unit (SAU) perform sequential access to the bitmap in SRAM. In each cycle, 4 bytes are loaded from SRAM into register A, while the current access address, which increments by 1 every cycle, is stored in register B. For an incoming lookup requests, the addresses are first stored in a buffer. In each cycle, addresses in this buffer are compared with the address in register B. If some addresses match the register B, the corresponding bit of data from register A is read to decide whether these addresses need to be output. If there is no match, the input addresses remains in the request buffer.

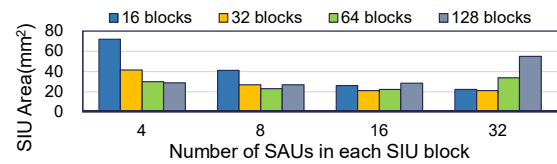
This approach suffers from obvious latency and area overhead issues. Due to the large size of the bitmap, sequentially accessing and traversing the entire bitmap takes a significant amount of time, resulting in extremely high latency for the matching of some requests. The high latency in processing requests leads to a large number of requests accumulating in the buffer, causing substantial overhead for buffer capacity and associated wiring for address comparison. Our evaluation shows that, to achieve the target throughput, the buffer needs to accommodate 1,059,584 memory access requests, with the total area for buffers and address comparison reaching  $418\text{mm}^2$ , which is impractical.

**Parallel Matching.** To mitigate this issue, we increase the number of SAUs and all memory access requests are compared with them in parallel. These SAUs access multiple SRAM banks in parallel at regular intervals. Thus that the latency for matching is largely reduced. Since the access to SRAM is sequential, ensuring that the initial access addresses of different SAUs are not in the same bank prevents bank conflicts.

However, parallel comparisons with multiple SAUs have led to a multiplicative increase in the number of comparators, and a high-radix crossbar is also required to connect the SAUs with the memory access requests in the request buffer. Our evaluation shows that to achieve the throughput target, the total area for the comparison network is  $123\text{mm}^2$ , which accounts for approximately 52.37% of the total chip area.



**Figure 10: Architecture of Sparse Index Unit (SIU). (a) Overall architecture of SIU; (b) a processing example of SIU block.**



**Figure 11: Area of SIU under different configurations to achieve the same target throughput.**

**Nearest grouping.** We observe that a memory access request only needs to match the address of any one SAU to return a result. Since each SAU accesses sequentially, we compare the address of each memory access request with the access addresses of each SAU to find the one that precedes and is closest to the request address (i.e., the SAU that can match the request the fastest). Based on this result, we group the access requests, with each group being compared only with the corresponding SAU's address. Figure 10(a) shows the architecture of a SIU block, where incoming memory access requests are compared with all addresses of SAUs to determine their groups, and subsequently enter the respective group request buffers. Each group request buffer corresponds to one SAU, and the memory requests within it only need to be compared with that SAU's address, thereby transforming the original  $mn \times m$  large comparison network into  $m$  independent  $n \times 1$  smaller comparison networks, where  $m$  is the number of SAUs, and  $n$  is the capacity of each group's request buffer. After this optimization, the comparison network's area is reduced to  $6.40mm^2$ , occupying only 2.73% of the total chip area.

After the above optimizations, 10(b) shows an processing example of a SIU block. Requests that arrive at the SIU block are allocated to the group request buffer corresponding to each SAU according to the nearest grouping strategy. The address corresponding to the 4 bytes in the bitmap accessed serially by the SAU in the current cycle is matched in parallel with the requests in the group request buffer. The matching addresses (e.g., requests 001F<sub>16</sub> and 0082<sub>16</sub>) are output based on the bitmap. If the bitmap access result is 1 (e.g., 001F<sub>16</sub>), the memory access request is stored in the output queue; otherwise, the request is filtered out directly.

**Hardware Implementation.** The aforementioned content of this section describes the structure and process of one SIU block. In the implementation of SIU, multiple blocks are contained in SIU to increase parallelism. Input requests are assigned to the corresponding SIU block based on the high bits of the addresses.

To choose an adequate configuration of SIU, we evaluate the area of SIU under different configurations and the results are shown in Figure 11. For different SIU configurations, we simulate the execution process of the SIU using a cycle-accurate simulator. Requests are input into SIU with the target throughput of SIU. By ensuring that no backpressure occurs during SIU execution, we determine the buffer size and the number of comparators in the SIU. Then, the area of the SIU is evaluated through EDA tools. According to the evaluation results, to achieve the lowest area, the SIU is divided into 32 blocks, each containing 16 SAUs. Each request buffer corresponding to one SAU needs to store up to 192 memory access requests, and the total area of the SIU is  $20.17mm^2$ . If SIU is divided into more blocks, the grouping crossbar incurs significant area overheads. If SIU is divided into fewer blocks, larger request buffers and more comparators are needed to prevent performance degradation due to backpressure.

Overall, compared to the naive crossbar approach, SIU increase throughput by  $5.88\times$ . Compared to scenarios without using SIU to filter out ineffective memory accesses, SIU achieves a  $7.54\times$  throughput improvement during the encoding stage, at the cost of 8.59% of the total chip area.

#### 4.5 MLP Units with Dynamic Shared Buffer

Following the aforementioned optimizations, the encoding stage is accelerated by  $7.54\times$ , making MLP units the new bottleneck. To improve the overall throughput, it is necessary to increase the number of MLP units. We find that the number of MLP units needs to be increased by at least  $4\times$  to match the throughput of the optimized encoding stage. However, since each MLP unit of Cambricon-R contains a large local buffer to store the training data of a ray, directly increasing the number of MLP units by  $4\times$  would require an additional total buffer capacity of 144 MB, significantly raising the cost of the chip. As illustrated in Figure 8, we propose a design with dynamic shared buffers that reduces the average buffer capacity required per MLP unit, so that we can largely increase the number of MLP units to improve performance.

**4.5.1 Shared Buffer.** Since the minimum unit to compute loss is a ray, each MLP unit performs MLP training with the granularity of all points along a single ray. Since the number of sampled points for each ray in the NSR algorithm varies, Cambricon-R designs the capacity of the local buffer in each MLP unit based on the maximum number of effective sampled points. This ensures support for fully fused on-chip training of the MLP. We observe that the number of sampled points for most rays is much less than the maximum, resulting in an average utilization rate of the local buffer being only 20.29%.

Based on this observation, we propose a design scheme where multiple MLP units share a buffer to enhance buffer utilization. The capacity of the shared buffer is greater than the maximum capacity required by a single ray, yet less than the total capacity if each of these MLP units had their own buffer. This design ensures that

even in extreme cases, the training of the ray with the maximum number of sampled points can be fully fused on the chip, while also avoiding the wastage of buffer capacity that occurs with most smaller rays.

The storage of the shared buffer is divided into two categories based on the type of data. Weights and the accumulated weight gradients, which are small in volume, are shared among all MLP units and stored in the weight buffer. In the forward stage, we use prefetching and merge memory accesses for the same address. In the backward stage, gradients computed by the MLP units are accumulated into the corresponding weight buffer using atomic operations. The activation data and their gradients for different rays are independent from each other. Each MLP unit occupy a larger volume for the activation and gradient data. And the volume varies for different rays. These data are stored in the activation buffer.

To simplify the management, the activation buffer is divided into multiple blocks. Each block is used to store training data for 32 points belonging to the same ray. Additionally, the processing of each ray is divided into multiple batches, each consisting of 32 points. This prevents conflicts when multiple MLP units access the shared activation buffer. Each group of MLP units is managed by a control unit, which contains a group of flag registers to maintain the states of each block of activation buffer. When processing a new batch of points, the MLP unit is first assigned a buffer block whose corresponding state register equals to 0 and changes it into 1. Then, the MLP unit record the index of this buffer block in a queue, thus it knows where to access each batch of points. After the finishing of each ray, the MLP unit change the state registers of the index in its queue into 0, and then free the queue.

**4.5.2 Dynamic Buffer Management.** The aforementioned scheme utilizes the dynamic sharing of the activation buffer among different MLP units to improve buffer utilization. Furthermore, we propose dynamic space management in the training of a single ray to reduce the capacity requirement of each block of activation buffer.

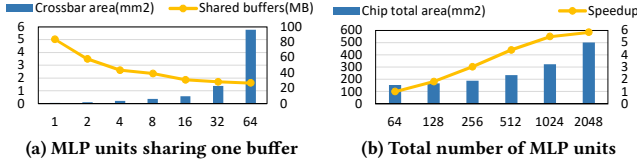
The baseline solution for performing MLP training is to allocate independent storage space to the activation and its gradient of all layers. However, each tensor in the MLP network has a different life period, which is an period from the first write of this data to the last read of it. We observe that the life period of some tensors begins after the end of some other tensors. For example, after computing the weight gradient and input gradient for the 5-th layer during the backward pass, the input activation of the 5-th layer can be deleted and the corresponding storage space can be released and reallocated to the input gradient of the 5-th layer. This inspire us that the later tensor can use the space released by the former tensor to save the use of memory space.

Based on this observation, we adopt dynamic buffer management, which dynamically allocates and releases storage space for different tensor within a block of the activation buffer. Experimental results show that this approach can reduce the size of each block of activation buffer by 49.42%.

**4.5.3 Design Space Exploration.** We conduct design space exploration to analyze the trade-off between performance and area. In Figure 12a, increasing the number of MLP units that share one buffer improves buffer utilization and reduces required total buffer

**Table 1: PSNR (dB) and SSIM comparison at the same modeling time**

| Algorithm   | Hardware     | Datasets |       |       |       |        |       |       |       |        |       |       |       |         |       |         |       |
|-------------|--------------|----------|-------|-------|-------|--------|-------|-------|-------|--------|-------|-------|-------|---------|-------|---------|-------|
|             |              | Fox      |       | Ficus |       | Hotdog |       | Mic   |       | Bonsai |       | Room  |       | Counter |       | Kitchen |       |
|             |              | PSNR↑    | SSIM↑ | PSNR↑ | SSIM↑ | PSNR↑  | SSIM↑ | PSNR↑ | SSIM↑ | PSNR↑  | SSIM↑ | PSNR↑ | SSIM↑ | PSNR↑   | SSIM↑ | PSNR↑   | SSIM↑ |
| Instant-NGP | Cambricon-R  | 32.6     | 0.915 | 31.0  | 0.856 | 35.6   | 0.964 | 34.7  | 0.930 | 26.0   | 0.804 | 27.5  | 0.873 | 22.9    | 0.770 | 24.5    | 0.810 |
|             | Cambricon-SR | 33.9     | 0.922 | 31.6  | 0.857 | 37.1   | 0.969 | 37.2  | 0.945 | 28.6   | 0.841 | 29.2  | 0.895 | 24.8    | 0.792 | 26.4    | 0.874 |
| Zip-NeRF    | Cambricon-R  | 21.9     | 0.694 | 23.1  | 0.916 | 25.1   | 0.835 | 24.8  | 0.928 | 17.8   | 0.556 | 17.2  | 0.662 | 17.1    | 0.570 | 16.2    | 0.443 |
|             | Cambricon-SR | 28.0     | 0.921 | 26.3  | 0.935 | 27.8   | 0.932 | 26.9  | 0.956 | 23.3   | 0.672 | 23.4  | 0.722 | 21.6    | 0.677 | 20.9    | 0.504 |
| K-Planes    | Cambricon-R  | 24.4     | 0.725 | 25.8  | 0.929 | 28.6   | 0.909 | 28.4  | 0.955 | 20.5   | 0.628 | 20.2  | 0.687 | 19.9    | 0.515 | 19.8    | 0.378 |
|             | Cambricon-SR | 29.7     | 0.827 | 26.4  | 0.946 | 33.2   | 0.958 | 30.6  | 0.967 | 22.6   | 0.670 | 23.3  | 0.711 | 21.8    | 0.569 | 21.8    | 0.419 |

**Figure 12: Design space exploration of the MLP module.**

size but significantly increasing crossbar area, especially at 64 units. Therefore, we adopt a design where 32 MLP units share one buffer. In Figure 12b, while the chip total area grows approximately linearly with more MLP units, the speedup shows diminishing returns at higher unit counts. Thus, 512 MLP units achieve a good balance between performance and cost.

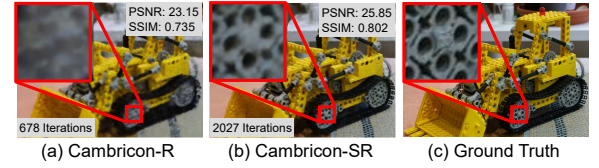
Using the above configuration, the dynamic shared buffer significantly optimizes resource utilization. Compared to Cambricon-R, it reduces the average buffer capacity per MLP unit by 85.31% while increasing utilization to 69.49%. To further increase throughput by 4×, we scale up the number of MLP units by 4×, yet the total buffer size of the MLP module decreases by 41%.

## 5 Experimental Evaluation

### 5.1 Methodology

**5.1.1 Hardware Implementation.** To evaluate Cambricon-SR, we develop a cycle-accurate simulator to measure runtime and hardware utilization. This simulator also tracks the number of DRAM access, enabling the calculation of energy consumption associated with off-chip memory accesses. Same as the baseline hardware configuration, we employ HBM2 as the off-chip memory, which offers a bandwidth of 1,555 GB/s. For accurate simulation of DRAM timing, Ramulator [20] is integrated into our simulator.

We also implement an RTL design of Cambricon-SR using Verilog to evaluate the area and energy consumption of Cambricon-SR. The RTL design is then synthesized using the Synopsys Design Compiler, targeting the TSMC 45nm technology node at a clock frequency of 750 MHz. The energy consumption of off-chip memory accesses is estimated at 3.9 pJ/bit [38]. Given that the baseline hardware operate on a 7nm node, we scale the area and power measurements of Cambricon-SR to the 7nm node, based on the method described in [44]. This approach is commonly used in research papers [26, 31].



**Figure 13: Rendered images at the same modeling time. Cambricon-SR achieves lower per-iteration runtime than Cambricon-R, completing more iterations within same modeling time.**

**5.1.2 Datasets & Benchmarks.** Eight datasets are involved in our evaluation, including Ficus, Hotdog, Mic, Fox, Bonsai, Room, Counter, and Kitchen. Ficus, Hotdog, and Mic are synthetic datasets from commonly-used NeRF-Synthetic [32]. Fox is a small-scale real-world dataset from Instant-NGP [36]. The other four datasets are larger-scale real-world datasets from Mip-NeRF-360 datasets [3].

We evaluate Cambricon-SR on three typical NeRF-based NSR algorithms: Instant-NGP [36], Zip-NeRF [4], and K-Planes [12]. In our experimental evaluation, the number of entries in each level of encoding tables and the MLP network configurations are same across the three algorithms. Instant-NGP and Zip-NeRF employ hash grid encoding, while K-Planes employs multi-planes encoding. Runtime and energy results in Section 5.2 are measured when the training quality reaches a PSNR of 25 following prior works [24].

**5.1.3 Baselines.** Cambricon-SR is compared against two baseline hardware: GPU and an NSR accelerator, Cambricon-R [42]. We choose NVIDIA A100 as the GPU baseline. For evaluations of Instant-NGP on the GPU, we execute open-source CUDA codes obtained from the official Instant-NGP repository [37]. It is specifically optimized in small matrix multiplications with tensor cores. However, the irregular fine-grained memory accesses during the encoding stage still leads to low utilization. This code is also the GPU baseline for many NSR accelerators [21, 24, 42, 55]. Zip-NeRF and K-Planes are evaluated in the Nerfstudio framework [45]. To fairly compare with Cambricon-R, we also implement Cambricon-R in a cycle-accurate simulator according to its paper.

### 5.2 Evaluation Results

**5.2.1 Modeling Quality.** We evaluate the modeling quality (PSNR and SSIM) on Cambricon-R and Cambricon-SR **when given 0.1 second to model each scene** (i.e., 10 scenes per second), as shown

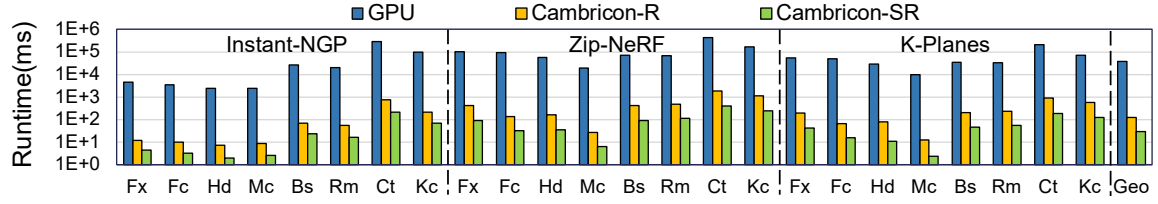


Figure 14: Runtime comparison.

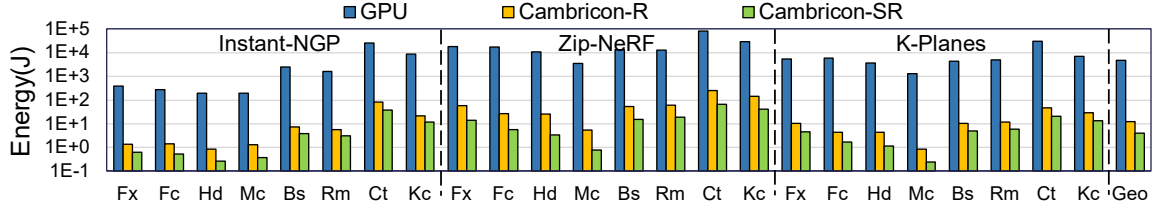


Figure 15: Energy comparison.

in Table 1. We do not compare against GPU since GPU could only complete approximately 5 iterations within 0.1 second, resulting in completely noisy outputs. To match the modeling quality of Cambricon-SR at 0.1 second per scene, GPU would require approximately 2 minutes of training time per scene. Although ST-NSR introduces a slight PSNR loss under the same number of iterations, the higher throughput of ST-NSR on Cambricon-SR enables more iterations to be performed under the same modeling time, which not only compensates for this loss but also achieves higher quality, as shown in Figure 13.

The PSNR and SSIM of the latter four datasets are lower than those of the first four. The reason is that the last four datasets represent large-scale real-world scenes that demand higher modeling speeds. Instant-NGP achieves better modeling quality than Zip-NeRF and K-Planes because its shorter per-iteration time allows more training iterations under the same modeling time.

Table 2: Area and Power of Cambricon-SR

|                           | Area (mm <sup>2</sup> ) | %      | Power (W) | %      |
|---------------------------|-------------------------|--------|-----------|--------|
| Sampling Module           | 2.64                    | 1.12   | 5.62      | 1.58   |
| Sparse Index Unit (SIU)   | 20.17                   | 8.59   | 53.81     | 15.09  |
| Encoding Module (w/o SIU) | 116.41                  | 49.57  | 189.00    | 53.01  |
| MLP Module                | 89.79                   | 38.23  | 97.48     | 27.34  |
| Update Module             | 1.10                    | 0.47   | 2.08      | 0.58   |
| Global Buffer             | 3.55                    | 1.51   | 4.48      | 1.26   |
| Other                     | 1.20                    | 0.51   | 4.10      | 1.15   |
| Total                     | 234.86                  | 100.00 | 356.57    | 100.00 |

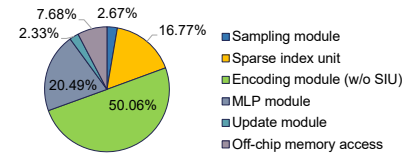


Figure 16: Energy breakdown.

**5.2.2 Performance.** Figure 14 shows the total training time on 8 datasets and the geometric mean of both our design and baselines. The results indicate that Cambricon-SR achieves 1259× and 4.12× speedup on average over GPU and Cambricon-R, respectively. The performance improvement of Cambricon-SR over Cambricon-R mainly comes from two aspects. On the one hand, the sparsification of the encoding table reduces the number of valid table accesses and invalid table accesses are filtered by SIU. On the other hand, by sharing buffers among different MLP units, the size of the buffer is decreased, allowing more compute units to be added.

As shown in Figure 14, the runtime for the first four datasets is significantly shorter than that of the latter four datasets. The main reason is that the latter four datasets are larger-scale and more iterations are needed to reach the same PSNR, as mentioned in Section 5.1.2. For instance, to achieve a PSNR of 25 in Instant-NGP, around 200 training iterations is required in the small-scale dataset Fox, while around 4,300 training iterations is required in the large-scale dataset Kitchen.

**5.2.3 Area and Energy Efficiency.** Table 2 shows the area and the power consumption of Cambricon-SR. Compared to Cambricon-R, Cambricon-SR achieves a speedup of 4.12×, while only increasing the total area by 43.0%. The additional area primarily comes from the units that handle the sparse table, including CAM, Sparse Update Unit, and SIU. More MLP units are employed in Cambricon-SR, but the total area of the MLP Module is less than that in Cambricon-R. The reason is that dynamic shared buffer in Cambricon-SR effectively reduces the capacity of buffers in MLP Module.

Figure 15 presents the energy consumption of Cambricon-SR and baselines for processing NSR training on 8 datasets. Compared with GPU and Cambricon-R, Cambricon-SR reduces the energy consumption by 1139× and 2.98×, on average. The reduction of energy consumption is mainly attributed to the decrease of encoding table accesses, which accounts for most of the energy consumption in Cambricon-R.



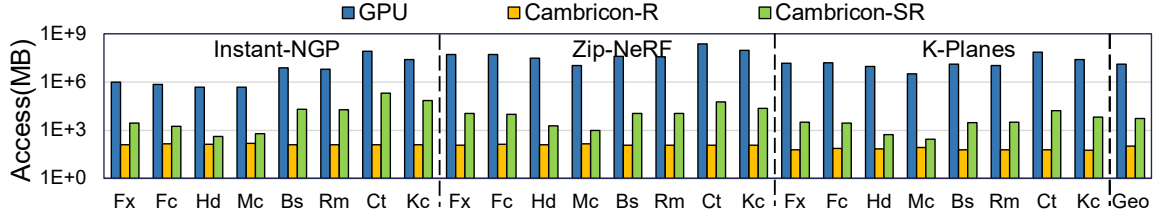


Figure 17: Off-chip access comparison.

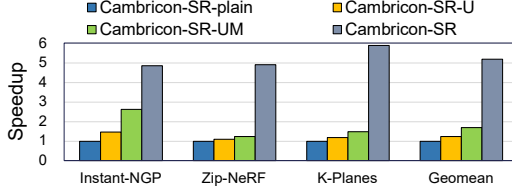


Figure 18: Ablation study.

Figure 16 presents the energy breakdown of Cambricon-SR. Most of the energy is consumed in MLP module and the encoding module. Only 7.68% of the total energy consumption is consumed in off-chip memory accesses, as Cambricon-SR nearly eliminates off-chip memory access during the encoding stage by storing the entire ST on-chip. Furthermore, the sparse update unit reduces off-chip memory accesses during the update stage by 6.06 $\times$ .

SIU accounts for 16.77% of the total energy consumption while occupying only 8.59% of the total area. The relatively high energy consumption of SIU is attributed to continuous sequential bitmap accesses with 70.7% of accessed data unused. The unused accesses contribute to 52.8% of SIU's energy consumption. Overall, sequential bitmap accesses in SIU result in only an 8.85% energy waste, but achieve a 7.54 $\times$  speedup in the encoding stage and a 3.07 $\times$  overall speedup. This is a worthwhile trade-off.

**5.2.4 Off-chip Memory Access.** Figure 17 compares the total amount of off-chip memory accesses of Cambricon-SR and baselines during the NSR training. Compared with GPU, both Cambricon-SR and Cambricon-R significantly reduce the off-chip memory accesses because they store most of the intermediate results on-chip and avoid the off-chip memory access of intermediate results. Cambricon-R eliminates nearly all off-chip memory accesses by storing the entire encoding table and all intermediate data during the training process on-chip. Cambricon-SR stores the sparse encoding table and table gradients on-chip, eliminating off-chip memory accesses during the forward and backward computation stages. However, in Cambricon-SR, off-chip access to the dense table is still required during the update stage of each training iteration. Therefore, more off-chip memory accesses are required for Cambricon-SR compared with Cambricon-R.

**5.2.5 Ablation Study.** As shown in Figure 18 and 19, we conduct experimental evaluations on four variants of the Cambricon-SR to analyze contributions of the proposed optimizations. All of four variants execute the ST-NSR algorithm.

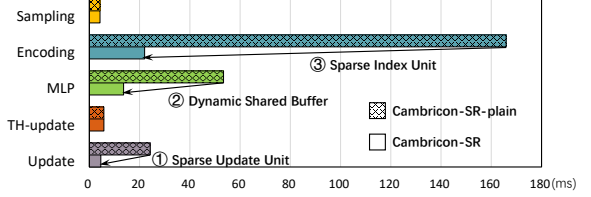


Figure 19: Runtime breakdown based on the dataflow in Figure 7. 'TH-update' refers to threshold update, while 'Update' includes updating MLP weights, ST and DT.

**The impact of the Sparse Update Unit (Figure 19①).** As the baseline implementation, Cambricon-SR-plain operates without the optimizations in Section 4. Compared to Cambricon-SR-plain, Cambricon-SR-U incorporates the Sparse Update Unit for efficient updates in ST, achieving a 1.24 $\times$  speedup over Cambricon-SR-plain. Unlike Cambricon-SR-plain, which accesses the entire DT each iteration, Cambricon-SR-U updates only changed entries, reducing the execution time of the table update stage by 5.28 $\times$ .

**The impact of the Dynamic Shared Buffer (Figure 19②).** Cambricon-SR-UM further integrates the Dynamic Shared Buffer to optimize buffer usage in the MLP module, achieving a 1.36 $\times$  speedup over Cambricon-SR-U. By reducing the average buffer usage per MLP unit by 85.3%, the Dynamic Shared Buffer allows for a 4 $\times$  increase in MLP units, resulting in a 3.92 $\times$  speedup in the MLP stage while reducing the total buffer size by 41%.

**The impact of the Sparse Index Unit (Figure 19③).** The complete Cambricon-SR introduces the Sparse Index Unit (SIU), achieving an additional 3.07 $\times$  speedup over Cambricon-SR-UM. SIU optimizes the encoding stage by filtering out invalid table access requests. With only 8.59% increase in area, SIU improves the throughput of the encoding stage by 7.54 $\times$ , further enhancing the performance.

We observe that the optimization effect of the sparse update unit and dynamic shared buffer in Zip-NeRF and K-Planes is less significant. The reason is that the workload of encoding table accesses is relatively large in Zip-NeRF and K-Planes. Consequently, the impact of optimizing the MLP stage and the update stage is limited, as the overall performance is constrained by the encoding stage.

## 6 Related Work

We review the related works on neural scene representation and neural rendering algorithms and hardware accelerators for neural scene representation and neural rendering.

**Neural scene representation and neural rendering algorithms.** Neural scene representation and neural rendering are first proposed by [33] in 2020. [33] proposes the basic idea and process of neural scene representation. However, it introduces a large amount of computations into the rendering process due to the large MLP. Many algorithms are proposed to solve this problem. Instant-NGP [36] introduces the encoding table to learn some of the representations so that the size of MLPs can be reduced. Zip-NeRF [4] combines the anti-aliasing features of Mip-NeRF 360 with the fast grid training of Instant-NGP, which not only improves the rendering quality but also effectively speeds up the training.

**Sparse NSR Algorithm.** Currently, many works have applied sparsification to NSR algorithms [13, 28, 36, 50]. [28] represents scenes using a sparse voxel octree format, allowing the rendering process to skip voxels that do not contain scene information. [13] represents a scene as a sparse voxel grid with spherical harmonics, which eliminates the need for neural networks and significantly improves training and rendering speed. [36] maintains an occupancy grid during training to record the occupancy of each grid cell, skipping sampled points within empty grid cells. These sparse algorithms perform sparsification during the sampling stage, skipping sparse sampled points. In contrast, our proposed sparse algorithm further sparsifies the encoding table during the encoding stage, allowing it to bypass parts of the dense sampled points when accessing sparse table entries.

**Hardware Accelerators for NSR.** Recent works have proposed several accelerators for both NSR modeling [24, 40, 42, 55] and rendering [11, 14, 16, 21, 22, 35, 39]. [21] proposes restricted hashing algorithm, which can avoid the irregular off-chip memory access by partitioning the hash table and loading only a portion of the table into on-chip memory at a time. [11] introduces a frame reusing algorithm, which exploits the similarity between adjacent frames during the rendering process. [24] proposes to split the NSR hash grid into 2 parts and reduce the grid size and update frequency for the less sensitive branch. [55] employs a Near Memory Processing (NMP) architecture to address the memory access bottleneck in the NSR processing. [42] proposes a ray-based execution flow to eliminate almost all of the off-chip memory accesses and an NoC to address the challenges of irregular memory access.

## 7 Conclusion

In this paper, we propose Cambricon-SR, an algorithm-hardware co-designed accelerator to improve NSR performance and quality. We propose a novel NSR algorithm with sparse encoding table, it achieves more than 80% of sparsity for the encoding table with negligible accuracy loss. To the best of our knowledge, this is the first NSR training algorithm with sparse encoding tables. Then, to efficiently eliminate the invalid memory access requests to the pruned entries, we propose the sparse index unit based on sequential SRAM access to address the challenge of irregularity memory access to the bitmap. It achieves a throughput increase of 7.54 $\times$  for the encoding stage with an additional area overhead of 8.59%. Third, we propose the dynamic shared buffer for the MLP units to reduce the buffer usage by 85.3% so that we can increase the number of the MLP units by 4 $\times$  with acceptable cost of on-chip buffers. We conduct evaluations on 8 typical scenes. The results show that, compared

with A100 GPU and Cambricon-R, Cambricon-SR achieves 1259 $\times$  and 4.12 $\times$  speedup, and reduces the energy consumption by 1139 $\times$  and 2.98 $\times$  for each training iteration of NSR, respectively. Thus, Cambricon-SR largely improves both the performance and quality compared with the baseline.

## Acknowledgments

This work is partially supported by the NSF of China (Grants No. 62302482, U22A2028, 62222214, 62341411, 62302478, 62302483, 6240073476, 62302480, 62302481), the Science and Technology Major Special Program of Jiangsu (Grants No. BG2024028), Strategic Priority Research Program of the Chinese Academy of Sciences, (Grants No. XDB0660200, XDB0660201, XDB0660202), CAS Project for Young Scientists in Basic Research (YSBR-029) and Youth Innovation Promotion Association CAS.

## References

- [1] Michal Adamkiewicz, Timothy Chen, Adam Caccavale, Rachel Gardner, Preston Culbertson, Jeannette Bohg, and Mac Schwager. 2022. Vision-only robot navigation in a neural radiance world. *IEEE Robotics and Automation Letters* 7, 2 (2022), 4606–4613.
- [2] Chong Bao, Yinda Zhang, Bangbang Yang, Tianxing Fan, Zesong Yang, Hujun Bao, Guofeng Zhang, and Zhaopeng Cui. 2023. Sine: Semantic-driven image-based nerf editing with prior-guided editing field. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 20919–20929.
- [3] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. 2022. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5470–5479.
- [4] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. 2023. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 19697–19705.
- [5] Arunkumar Byravan, Jan Humprik, Leonard Hasenclever, Arthur Brussee, Francesco Nori, Tuomas Haarnoja, Ben Moran, Steven Bohez, Fereshteh Sadeghi, Bojan Vujatovic, and Nicolas Heess. 2023. Nerf2real: Sim2real transfer of vision-guided bipedal motion skills using neural radiance fields. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 9362–9369.
- [6] Junyi Cao, Zhichao Li, Naiyan Wang, and Chao Ma. 2024. Lightning NeRF: Efficient Hybrid Scene Representation for Autonomous Driving. In *IEEE International Conference on Robotics and Automation (ICRA)*.
- [7] Timothy Chen, Preston Culbertson, and Mac Schwager. 2024. Catnips: Collision avoidance through neural implicit probabilistic scenes. *IEEE Transactions on Robotics* (2024).
- [8] You-Rong Chen, Chien-Chia Ho, Wei-Ting Chen, and Pei-Yin Chen. 2023. A Low-Cost Pipelined Architecture Based on a Hybrid Sorting Algorithm. *IEEE Transactions on Circuits and Systems I: Regular Papers* (2023).
- [9] Zhang Chen, Zhong Li, Liangchen Song, Lele Chen, Jingyi Yu, Junsong Yuan, and Yi Xu. 2023. Neurbf: A neural fields representation with adaptive radial basis functions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4182–4194.
- [10] Nianchen Deng, Zhenyi He, Jiannan Ye, Budmonde Duinkharjav, Praneeth Chakravarthula, Xubo Yang, and Qi Sun. 2022. Fov-nerf: Foveated neural radiance fields for virtual reality. *IEEE Transactions on Visualization and Computer Graphics* 28, 11 (2022), 3854–3864.
- [11] Yu Feng, Zihan Liu, Jingwen Leng, Minyi Guo, and Yuhao Zhu. 2024. Cicero: Addressing Algorithmic and Architectural Bottlenecks in Neural Rendering by Radiance Warping and Memory Optimizations. *arXiv preprint arXiv:2404.11852* (2024).
- [12] Sara Fridovich-Keil, Giacomo Meanti, Frederik Rahbæk Warburg, Benjamin Recht, and Angjoo Kanazawa. 2023. K-planes: Explicit radiance fields in space, time, and appearance. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12479–12488.
- [13] Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. 2022. Plenoxels: Radiance fields without neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5501–5510.
- [14] Yonggan Fu, Zhifan Ye, Jiayi Yuan, Shun Yao Zhang, Sixu Li, Haoran You, and Yingyan Lin. 2023. Gen-nerf: Efficient and generalizable neural radiance fields via algorithm-hardware co-design. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–12.

- [15] Jianfei Guo, Nianchen Deng, Xinyang Li, Yeqi Bai, Botian Shi, Chiyu Wang, Chenjing Ding, Dongliang Wang, and Yikang Li. 2023. Streetsurf: Extending multi-view implicit surface reconstruction to street views. *arXiv preprint arXiv:2306.04988* (2023).
- [16] Donghyeon Han, Junha Ryu, Sangyeob Kim, Sangjin Kim, and Hoi-Jun Yoo. 2023. 2.7 MetaVRain: A 133mW Real-Time Hyper-Realistic 3D-NeRF Processor with 1D-2D Hybrid-Neural Engines for Metaverse on Mobile Devices. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 50–52.
- [17] Ayaan Haque, Matthew Tancik, Alexei A Efros, Aleksander Holynski, and Angjoo Kanazawa. 2023. Instruct-nerf2nerf: Editing 3d scenes with instructions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 19740–19750.
- [18] Benran Hu, Junkai Huang, Yichen Liu, Yu-Wing Tai, and Chi-Keung Tang. 2023. Nerf-rpn: A general framework for object detection in nerfs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 23528–23538.
- [19] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics* 42, 4 (2023), 1–14.
- [20] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer architecture letters* 15, 1 (2015), 45–49.
- [21] Junseo Lee, Kwansook Choi, Jungi Lee, Seokwon Lee, Joonho Whangbo, and Jaewoong Sim. 2023. NeuRex: A Case for Neural Rendering Acceleration. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [22] Chaojian Li, Sixu Li, Yang Zhao, Wenbo Zhu, and Yingyan Lin. 2022. RT-NeRF: Real-time on-device neural radiance fields towards immersive AR/VR rendering. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [23] Ke Li, Susanne Schmidt, Tim Rolf, Reinhard Bacher, Wim Leemans, and Frank Steinicke. 2024. Magic nerf lens: Interactive fusion of neural radiance fields for virtual facility inspection. *Frontiers in Virtual Reality* 5 (2024), 1377245.
- [24] Sixu Li, Chaojian Li, Wenbo Zhu, Boyang Yu, Yang Zhao, Cheng Wan, Haoran You, Huihong Shi, and Yingyan Lin. 2023. Instant-3d: Instant neural radiance field training towards on-device ar/vr 3d reconstruction. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [25] Tianye Li, Miroslava Slavcheva, Michael Zollhoefer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, S. Lovegrove, Michael Goesele, and Zhaoyang Lv. 2021. Neural 3D Video Synthesis. *ArXiv abs/2103.02597* (2021).
- [26] Zheng Li, Sorous Ghodrati, Amir Yazdanbakhsh, Hadi Esmailzadeh, and Mingu Kang. 2022. Accelerating attention through gradient-based learned runtime pruning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 902–915.
- [27] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. 2021. Neural Scene Flow Fields for Space-Time View Synthesis of Dynamic Scenes. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 6494–6504.
- [28] Lingjie Liu, Jiatao Gu, Kyaw Zaw Lin, Tat-Seng Chua, and Christian Theobalt. 2020. Neural Sparse Voxel Fields. *NeurIPS* (2020).
- [29] Stephen Lombardi, Tomas Simon, Jason M. Saragih, Gabriel Schwartz, Andreas M. Lehrmann, and Yaser Sheikh. 2019. Neural volumes. *ACM Transactions on Graphics (TOG)* 38 (2019), 1–14.
- [30] Dominic Maggio, Marcus Abate, Jingnan Shi, Courtney Mario, and Luca Carlone. 2023. Loc-nerf: Monte carlo localization using neural radiance fields. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 4018–4025.
- [31] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremiah Kim, Daberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alser, Rachata Ausavarungrinun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. 2022. GenStore: a high-performance in-storage processing system for genome sequence analysis. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 635–654.
- [32] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *ECCV*.
- [33] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2021. NeRF: Representing scenes as neural radiance fields for view synthesis. *Commun. ACM* 65, 1 (2021), 99–106.
- [34] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378* (2021).
- [35] Muhammad Husnain Mubarak, Ramakrishna Kanungo, Tobias Zirr, and Rakesh Kumar. 2023. Hardware acceleration of neural graphics. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–12.
- [36] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)* 41, 4 (2022), 1–15.
- [37] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. The Official Instant-NGP Repository. <https://github.com/NVlabs/instant-ngp>.
- [38] Mike O'Connor, Niladrish Chatterjee, Donghyuk Lee, John Wilson, Aditya Agrawal, Stephen W. Keckler, and William J. Dally. 2017. Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems. In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 41–54.
- [39] Chaolin Rao, Huangjie Yu, Haochuan Wan, Jindong Zhou, Yueyang Zheng, Minye Wu, Yu Ma, Anpei Chen, Binzhe Yuan, Pingqiang Zhou, Xin Lou, and Jingyi Yu. 2022. Icarus: A specialized architecture for neural radiance fields rendering. *ACM Transactions on Graphics (TOG)* 41, 6 (2022), 1–14.
- [40] Junha Ryu, Hankyul Kwon, Wonhoon Park, Zhiyong Li, Beomseok Kwon, Donghyeon Han, Dongseok Im, Sangyeob Kim, Hyungnam Joo, and Hoi-Jun Yoo. 2024. 20.7 NeuGPU: A 18.5 mJ/Iter Neural-Graphics Processing Unit for Instant-Modeling and Real-Time Rendering with Segmented-Hashing Architecture. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 67. IEEE, 372–374.
- [41] Farhad Shafai, Kenneth J. Schultz, GF Randall Gibson, Armin G. Bluschke, and David E. Somppi. 1998. Fully parallel 30-MHz, 2.5-Mb CAM. *IEEE Journal of Solid-State Circuits* 33, 11 (1998), 1690–1696.
- [42] Xinkai Song, Yuanbo Wen, Xing Hu, Tianbo Liu, Haoxuan Zhou, Husheng Han, Tian Zhi, Zidong Du, Wei Li, Rui Zhang, Chen Zhang, Lin Gao, Qi Guo, and Tianshi Chen. 2023. Cambricon-R: A Fully Fused Accelerator for Real-Time Learning of Neural Scene Representation. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1305–1318.
- [43] Pratul P. Srinivasan, Boyang Deng, Xiuming Zhang, Matthew Tancik, Ben Mildenhall, and Jonathan T. Barron. 2021. Nerv: Neural reflectance and visibility fields for relighting and view synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7495–7504.
- [44] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration* 58 (2017), 74–81.
- [45] Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Justin Kerr, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, David McAllister, and Angjoo Kanazawa. 2023. Nerfstudio: A Modular Framework for Neural Radiance Field Development. In *ACM SIGGRAPH 2023 Conference Proceedings (SIGGRAPH '23)*.
- [46] Peng Wang, Yuan Liu, Zhaoxi Chen, Lingjie Liu, Ziwei Liu, Taku Komura, Christian Theobalt, and Wenping Wang. 2023. F2-nerf: Fast neural radiance field training with free camera trajectories. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4150–4159.
- [47] Suttisak Wizadwongsa, Pakkapon Phongthawee, Jiraphon Yenphraphai, and Supasorn Suwajanakorn. 2021. Nex: Real-time view synthesis with neural basis expansion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8534–8543.
- [48] Chenming Wu, Jiada Sun, Zhelun Shen, and Liangjun Zhang. 2023. Mapnerf: Incorporating map priors into neural radiance fields for driving view simulation. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 7082–7088.
- [49] Zhijie Wu, Yuhe Jin, and Kwang Moo Yi. 2023. Neural fourier filter bank. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14153–14163.
- [50] Xiufeng Xie, Riccardo Gherardi, Zhihong Pan, and Stephen Huang. 2023. Hollownerf: Pruning hashgrid-based nerfs with trainable collision mitigation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3480–3490.
- [51] Ziyang Xie, Junge Zhang, Wenye Li, Feihu Zhang, and Li Zhang. 2023. S-NeRF: Neural Radiance Fields for Street Views. In *International Conference on Learning Representations (ICLR)*.
- [52] Honghui Yang, Sha Zhang, Di Huang, Xiaoyang Wu, Haoyi Zhu, Tong He, Shixiang Tang, Hengshuang Zhao, Qibo Qiu, Binbin Lin, Xiaofei He, and Wanli Ouyang. 2024. Unipad: A universal pre-training paradigm for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 15238–15250.
- [53] Yu-Jie Yuan, Yang-Tian Sun, Yu-Kun Lai, Yewen Ma, Rongfei Jia, and Lin Gao. 2022. Nerf-editing: geometry editing of neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 18353–18364.
- [54] Xiuming Zhang, Pratul P. Srinivasan, Boyang Deng, Paul Debevec, William T. Freeman, and Jonathan T. Barron. 2021. Nerfactor: Neural factorization of shape and reflectance under an unknown illumination. *ACM Transactions on Graphics (TOG)* 40, 6 (2021), 1–18.
- [55] Yang Katie Zhao, Shang Wu, Jingqun Zhang, Sixu Li, Chaojian Li, and Yingyan Celine Lin. 2023. Instant-NeRF: Instant On-Device Neural Radiance Field Training via Algorithm-Accelerator Co-Designed Near-Memory Processing. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [56] Zhenxin Zhu, Yuntao Chen, Zirui Wu, Chao Hou, Yongliang Shi, Chuxuan Li, Pengfei Li, Hao Zhao, and Guyue Zhou. 2023. Latitude: Robotic global localization with truncated dynamic low-pass filter in city-scale nerf. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 8326–8332.